Recommendation Data Pipeline: Final Report

End-to-end pipeline from raw data to evaluated recommenders, orchestrated and tracked. Author: Sunit Kamble

1. Problem and data

The goal is to recommend relevant items to users of an online store from past interactions and item information, using a clean, modular and reproducible pipeline. Because the focus is pipeline quality, I generated **documented synthetic data** (1,500 users, 600 items, 8 categories, about 58k ratings from 2023-2024). Ratings come from latent user/item factors plus category structure, item bias and noise, and users mostly interact with popular items (popularity-biased exposure), as in real shops. Realistic dirt is injected on purpose (nulls, duplicates, invalid ratings and timestamps, wrong types, orphan keys) so the validation stage is exercised. A loader for MovieLens or Amazon Reviews can replace the generator without changing later stages.

2. Architecture

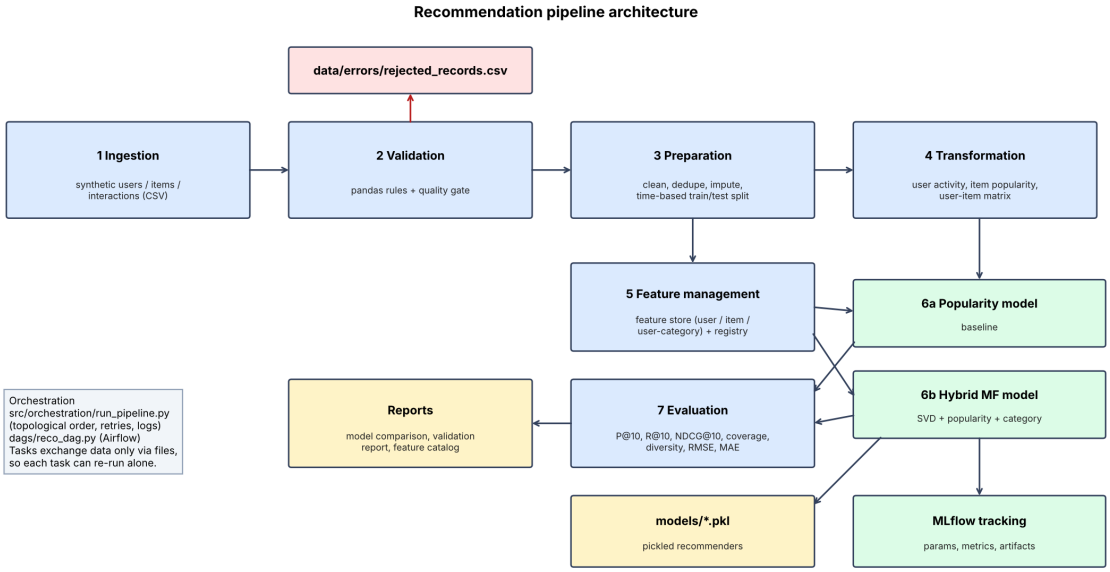


Figure 1. Eight stages; tasks exchange data only through files, so each can be re-run on its own.

3. Design decisions and tools

- **Validation (pandas rules):** missing values, duplicates, wrong types, ratings outside 1-5, unparsable or future timestamps and orphan user/item keys. Rejected rows go to *data/errors/rejected_records.csv* with a reason, and a quality gate stops the run if more than 10% of any table is rejected.
- **Preparation and transformation:** standardised text, imputed missing ages, kept the latest rating per user-item pair, and used a **time-based split** (latest 20% of time is test) so the model never sees the future. Derived datasets: user activity summary, item popularity and a sparse user-item matrix.
- **Feature management:** a small file-based feature store builds user, item and user-category features from **train data only** and exposes one read path (*get_user_features, get_item_features, interaction_features*) used by both training and prediction, with defaults for cold-start users/items. A registry documents every feature (catalog in the repository README).
- **Models:** (a) popularity baseline ranked by positive-rating count; (b) hybrid recommender: SVD matrix factorisation on rating residuals (user mean plus shrunken item bias plus 16 latent factors), blended with item popularity and user-category affinity from the feature store.
- **Tracking and orchestration:** MLflow (SQLite backend) logs parameters, metrics and artifacts. A dependency-aware Python runner (graphlib) with retries and logs runs the DAG, and *dags/reco_dag.py* expresses the same graph for Apache Airflow. Runs are deterministic (fixed seed).
- **Tools:** Python, pandas, NumPy, SciPy, scikit-learn, MLflow, matplotlib, pytest, reportlab.

4. Results

Validation. All tables passed the quality gate. Interaction rejections are mostly orphan keys (1,706 item, 470 user; some cascade from rejected parent rows), duplicates (562) and invalid ratings/timestamps/types (about 295 each). Split: cutoff 2024-09-21, 43,369 train rows and 10,843 test rows.

Table	Input rows	Valid rows	Rejected	Rate
users	1505	1493	12	0.8%
items	603	591	12	2.0%
interactions	58715	54212	4503	7.7%

Model comparison (K = 10, 1,360 test users, relevant = rating of 4 or more):

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
Popularity	0.0983	0.2094	0.1897	0.0626	0.9348	0.8890	0.7263
Hybrid MF	0.1176	0.2230	0.2138	0.1540	0.8824	0.8335	0.6578
Change	+19.6%	+6.5%	+12.7%	+146.0%	-5.6%	-6.2%	-9.4%

The hybrid model improves ranking quality (precision +19.6%, NDCG +12.7%), recommends a much wider part of the catalogue (coverage 0.063 to 0.154) and predicts ratings better (RMSE 0.889 to 0.834). The price is slightly lower diversity, because its lists lean on popular items in the preferred category.

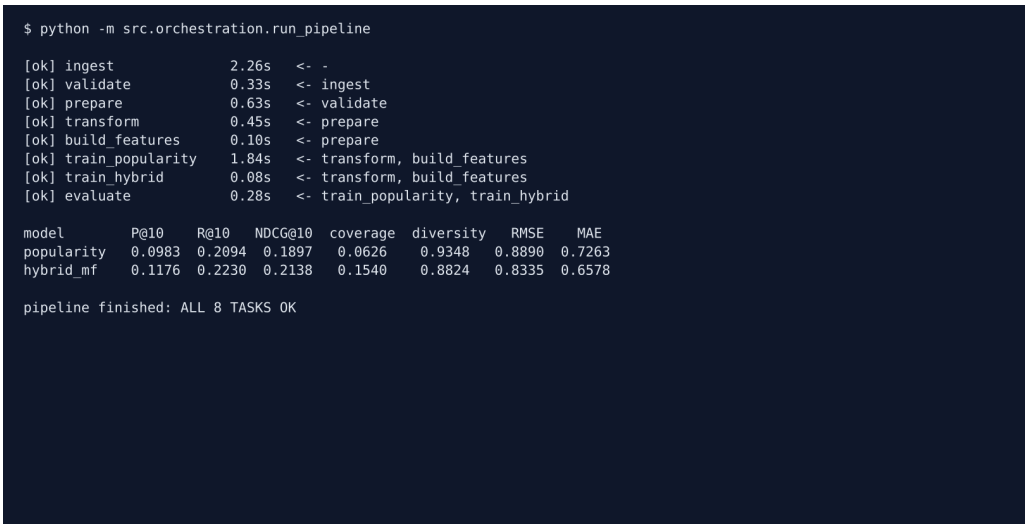


Figure 2. Successful end-to-end run, rendered from run_status.json and model_comparison.csv (raw terminal and MLflow screenshots are in reports/screenshots).

5. Limitations and future work

- **Synthetic data:** structure and popularity bias are assumed, so absolute scores will differ on real data (MovieLens, Amazon Reviews).
- **Tuning:** the hybrid blend weights came from a small grid scored on the test set, so the gain is slightly optimistic. A validation window taken from the end of training data should be used in production.
- **Validation:** custom pandas rules are simple and transparent; Pandera or Great Expectations would add reusable, versioned expectation suites.
- **Scale and storage:** CSV files, a dense per-user score matrix and a local MLflow store suit this size only. Larger data needs Parquet/a database, approximate nearest-neighbour retrieval and a hosted tracking server.
- **Orchestration:** the Python runner is the tested path; the Airflow DAG reuses its task graph but was not executed here. Future work includes implicit-feedback models (ALS, LightFM), online evaluation and monitoring for data drift.